Attachment 5

The 1st Global College Student Life Science Challenge Code Submission Requirements

AI Models and Code · Submission Standards and Reproducibility Requirements

To ensure standardized, fair, and reproducible evaluation of AI models and code, and to facilitate standardized verification of entries by the competition evaluation and presentation platform, these code-submission instructions have been established. All teams whose projects involve artificial-intelligence models or computational methods shall submit their code, models, and operating documentation in accordance with these requirements. These are general requirements; individual tracks may impose supplementary rules. Details such as file naming and field formats shall follow the track-specific implementation rules, Submission Template, and Intellectual Property Instructions published on the official competition website.

I. Code Execution Environment

Participants shall establish a fully operational algorithm training and design/inference environment on the computing platform provided by the competition, or in a local environment with the configuration clearly specified, and submit the following:

• Algorithm code: Follow standardized coding practices and comment the core code. The README must provide a complete list of dependency-library versions required for environment configuration and the command line needed to run the program. Submit the algorithm source code, which must run normally in the configured environment.

• Algorithm model: Provide the final model version produced by algorithm training, including model weights or instructions for invoking the model. If only an open-source pretrained model or third-party model is used, or if no training/fine-tuning has been performed, truthfully state the model and version used, invocation parameters, technical route, and the project’s actual innovative contribution. Whether the model was self-trained shall not, by itself, determine compliance.

• Demonstration case: Provide one complete executable example in the code environment covering “training/design → optimization → inference/screening,” together with small-scale example data or instructions for obtaining the data.

II. Directory Structure and Required Files

To facilitate testing and verification by the competition organizers, participants shall ensure that the submitted code repository or compressed package includes, but is not limited to, the following components. The recommended directory structure is:

```
project/
■■■ README.md             # Project description, environment setup, commands,
■                         # input/output, and explanation of results
■■■ requirements.txt      # Dependencies and required versions
■                         # (or environment.yml / Dockerfile)
■■■ data/                 # Data description and acquisition method
■                         # (do not upload restricted data; state source/license)
■■■ src/                  # Core source code and comments on core logic
■■■ models/               # Final model weights or download/invocation instructions
■■■ notebooks/            # Executable Jupyter Notebook demonstrating reproducibility
■■■ train.py              # One-command training script for self-trained models
■■■ design.py / screen.py # One-command design/generation or virtual-screening script
■■■ predict.py            # One-command generation of final candidate-results file
■■■ results/              # Example outputs and candidate lists
■■■ logs/                 # Training/validation logs and random-seed records
```

Required files and scripts:

(1) requirements.txt: Python dependencies and their versions shall be specified in requirements.txt, environment.yml, or a dependency lock file. The Python interpreter, operating system, CUDA, drivers, system libraries, and hardware requirements shall be described in README, environment.yml, or Dockerfile.

(2) Main execution entry point: Every project shall provide a main execution entry point capable of reading the required input and generating the final results file, such as run.sh, main.py, predict.py, design.py, or a functionally equivalent method. File names are not uniformly prescribed, but the complete command must be stated in the README.

(3) Training entry point: Projects that train or fine-tune their own models shall provide a training entry point, configuration files, data-split description, training logs, and the model weights intended for evaluation. Projects that did not perform training or fine-tuning are not required to submit a training script.

(4) Jupyter Notebook: At least one executable Notebook should be provided to demonstrate key processes. If the main execution entry point can already fully reproduce the workflow, a Notebook is not mandatory for every project.

III. Reproducibility and Traceability Requirements

To ensure that results are authentic and reproducible, the code and model attachments shall contain information enabling verification of originality and reproducibility:

• Data: State dataset sources and licenses, data-cleaning and preprocessing records, and the method used to split training/validation/test sets. Fully disclose the names, versions, acquisition dates, and purposes of all pretrained, training, validation, testing, fine-tuning, and external data, and explain deduplication and data-leakage prevention measures. For unpublished hidden evaluation sets, participating teams are responsible for truthful disclosure and reasonable preventive measures; the organizers are responsible for construction, confidentiality, and overlap checking of hidden evaluation sets.

• Training: Provide model architecture and key parameters, training and validation logs, random-seed settings, hardware used, and runtime.

• Model: Provide final model weights or platform operation records, together with a Model Card describing scope of application, inputs/outputs, and known limitations.

• Results: Provide predicted results for candidate molecules/tools/vectors, ranking logic, and necessary uncertainty analysis.

• Third-party models/platforms: When open-source models, third-party software, or commercial platforms are used, state their names, versions, invocation time, input information, and key parameters, and indicate sources and usage licenses. When APIs or commercial platforms are used, retain necessary prompts, invocation parameters, inputs/outputs, and invocation logs.

IV. Final Results File

The main execution entry point shall be capable of generating a standardized final results file in one step, serving as the candidate molecule/tool/vector list for verification by the organizers:

• Naming and format: Follow the Candidate List/Submission Template published on the official competition website. Unless otherwise specified, the default output shall be results.xlsx conforming to the template requirements, or results.csv encoded in UTF-8.

• Required fields: At minimum, include candidate ID, competition track, candidate sequence/SMILES/structure or component information, key predicted indicators (such as binding affinity, activity, editing efficiency, or delivery efficiency), corresponding model and run version,

and remarks. Track-specific submission templates shall further specify field names, data types, units, missing-value rules, number of candidates, ranking method, and numerical precision.

• Structure files: For projects involving three-dimensional structures, attach structure files such as PDB/CIF and indicate the corresponding file names in the candidate list. Each track shall specify naming rules, coordinate units, chain numbering, protonation state, and other necessary standards.

V. Special Circumstances

• Use of open-source pretrained models only: If no training or fine-tuning was performed, the report and README shall state the model name, version, invocation method, parameter settings, result-generation workflow, and the project's innovative contribution, and shall provide a main execution entry point capable of generating the final results file.

• Non-self-trained models or traditional approaches: Projects using models that were not self-trained, or traditional/non-deep-learning approaches, shall explain this in the report and provide code, scripts, workflows, or other functionally equivalent materials that allow verification of the core computational process and generation of the final results.

• Restricted data or resources: Do not upload raw restricted data. However, provide the data source, acquisition method, licensing information, and a complete reproducible workflow.

VI. Code Standards and Intellectual Property

• Standardization and robustness: Code should have a clear structure, standardized naming, and comments on core logic. Avoid hard-coded absolute paths; use relative paths or configuration files. Fix random seeds as far as possible to ensure reproducibility, and state expected runtime and resource requirements.

• Originality and compliance: Ensure the originality of the work and do not infringe others' intellectual property rights, data rights, or trade secrets. When public datasets, open-source models, or third-party platforms are used, accurately identify their sources, versions, and methods of use.

• Open-source sharing: Teams are encouraged, while protecting necessary intellectual property and complying with data licenses, to submit models, code, design workflows, and data to the competition's public platform. Results with intellectual-property protection value may be publicly shared after patent applications or achievement registration have been completed.

VII. Authenticity Verification

The competition reserves the right to run and verify submitted code and models. The evaluation and presentation platform will configure the environment according to the README and requirements.txt, run scripts such as predict.py, and verify consistency between the generated results and the submitted candidate list. During the defense, teams may be questioned regarding code logic, data sources, data-leakage prevention, and parameter settings. If data fabrication, non-reproducible results, or results that cannot be reasonably traced are identified, the project's competition or award eligibility will be revoked.